WORKSHOP: RAGNARÖK

Complete Instruction & Service Manual

Everything the app does, page by page — how the pieces connect, what to do when something breaks, and the full technical/deployment reference for keeping it running.

Manual version	Compiled July 2026
Application	Workshop: Ragnarök — auto shop management system
Covers	Every page and feature, a troubleshooting guide, and a full technical/service manual

Table of Contents

PART I

User Manual

1. Getting Started & Logging In

The app opens on a login screen with an animated 3D cat background ("Cooper" and "Roscoe"), purely decorative — their heads track your mouse cursor. The first time you open the app in a browser session, a one-time boot splash screen plays; click anywhere to skip it.

To log in, enter your username and password. There is a show/hide toggle on the password field. On success, the page fully reloads and drops you on the Dashboard.

> Special mode: if the app is opened somewhere it can't reach the real backend (for example, directly on localhost, or inside Google AI Studio's preview), it automatically logs you in as a fake admin account so the interface can still be reviewed. A small floating "Log In With Real Account" button is the only sign this is happening — click it to get back to a real, data-connected session.

Every page shares the same shell: the left sidebar for navigation, a top header (hidden only on the Video Editor), a per-page zoom control (70%-150%, remembered per page), and a floating AI chat bubble in the bottom-right corner that follows you everywhere. Some pages also have a custom background image or video, which you can set yourself in Settings.

2. Dashboard

Your landing page after login. An animated hero banner sits at the top, followed by four stat cards — Customers, Jobs, Vehicles, and Manuals — each of which jumps straight to that section when clicked.

Below that is the Quick Diagnostic Library Lookup: a single search box that searches both your vehicle records and the canned repair-procedure articles in the Manual Library at once. It understands common shorthand (e.g., typing "chevy" matches Chevrolet).

Three columns finish the page — Recent Repair Orders, Upcoming Appointments, and Recent Clients — each a shortcut into the full corresponding page. The Dashboard itself has no editable fields; it's a jumping-off point, not a place you enter data.

3. Customers

Your customer relationship management (CRM) directory. The list view has a search box, a tag filter, an "Export CSV" button (exports whatever the current filter shows, not always the full list), and an "Add Customer" button. Each row has quick actions to email, edit, or delete that customer (deleting always asks you to confirm first).

Opening a customer shows their profile: a contact card with clickable phone/email links, their assigned Tags (add or remove right on this screen), a grid of every vehicle they own (clicking one jumps to that vehicle's page), and their full repair-order history (clicking a job jumps to Jobs).

The Add/Edit Customer form lets you assign tags inline, and even has a small "Create New Tag" form built right in so you never have to leave the modal to make a new tag. There is also a separate "Manage Tags" screen for renaming, recoloring, or deleting tags shop-wide — deleting a tag warns you that it will remove that tag from every customer it's attached to.

4. Vehicles

The fleet registry — every vehicle any customer owns. A vehicle's profile page has several handy shortcuts: "Find Manual" jumps straight to the Manual Library pre-searched for that exact vehicle; a "Quick Sync" lets you update odometer reading and notes right there with no popup; "Find Nearby Parts" opens a Google Shopping search for parts near your shop's zip code (it asks for your shop's zip once, then remembers it in Settings).

Each vehicle page also shows the linked customer (read-only card), a list of manuals/procedures you've bookmarked to that vehicle, and a full Maintenance & Service Log where you can add, edit, or delete service history entries — saving one automatically updates the vehicle's current mileage.

To register a new vehicle, you pick the owning customer (required), then fill in year/make/model, VIN, color, engine, purchase and current mileage, and any notes.

> If you notice a leftover "Garage" concept anywhere in old notes or code comments, it isn't a real, reachable page in the app today — the Vehicles page replaced it. Don't go looking for a separate Garage tab; it doesn't exist in the current navigation.

5. Jobs / Work Orders

The busiest page in the app — this is where actual repair work gets tracked start to finish. The list view has status tabs (All / Pending / In Progress / Complete / Cancelled) each showing a live count, RUSH badges for priority jobs, and warnings when a job is missing its estimated hours. A "Services Catalog" button lets you manage your shop's standard service offerings (deleting one from the catalog doesn't affect any past ticket that already used it).

Opening a single work order (a "ticket") gives you the full toolkit:

- **Edit** — change any of the ticket's core details.
- **Print Invoice** — opens a clean, letterhead-style print view.
- **Download PDF** — generates a PDF invoice in your browser with your shop logo, line items, photos, and tax.
- **Pay Now** — sends the customer to a Stripe checkout page; this button disappears automatically once the ticket is paid.
- **Customer Portal Link** — generates (or regenerates) a private link you can email or copy to the customer so they can view and approve their own ticket online.
- **Notes** — free-text notes with file/photo attachments; attachments can be deleted individually and photos open in a full zoomable lightbox.
- **Parts** — add parts from your inventory (with a markup preview) or type in a custom one-off part, plus a "Find Nearby Price" shortcut.
- **Services** — add from your services catalog or a custom line item, including overage-hour costs if a job runs long.
- **Before/After Photos** — separate photo galleries with captions for each vehicle.
- **Status buttons** — four one-click buttons instantly update the ticket's status.

The running total (parts + services + labor + tax) updates live as you edit. When creating or editing a ticket, labor cost is automatically calculated from the estimated hours times your shop's labor rate (set in Settings), and you can mark a ticket Rush or Standard priority and flag whether the customer has approved the estimate.

6. Inventory

Tracks every part your shop stocks. The top of the page shows four metrics: total parts on hand, how many are low-stock, total asset value, and potential profit margin. Below that, you can filter by category or search by name/part number.

Every part row lets you adjust stock up or down with a reason (permanently logged for your records), plus normal edit/delete. When you add or edit a part, the sell price recalculates automatically from cost times your markup percentage.

The standout feature here is **Invoice Upload & AI Parsing**: drag and drop (or photograph) a batch of supplier invoices, click "Parse Invoice," and the built-in AI (Google Gemini) reads the invoice and extracts every line item automatically. A review screen lets you edit anything it got wrong and choose, per line, whether to create a brand-new part or add stock to an existing one (it tries to auto-match by part number or name first). One "Import" button commits everything you've checked, and the original receipt image is archived with a summary of what was imported. A separate Receipts Archive screen lets you search, filter by date, edit, or delete past receipts at any time.

7. Calendar

Month, Week, or Day scheduling views. A "Subscribe" button reveals a private iCal feed URL for your shop, so you can pipe your Ragnarök schedule into any calendar app that supports subscribing to a feed.

Appointments are color-coded by the status of the job they're linked to, and a daily-capacity bar shows how booked each day is. You can drag and drop an appointment to instantly reschedule it. An "Unscheduled Jobs" sidebar lists work orders that don't have an appointment yet — click one to schedule it in one step.

When creating an appointment you can link it to a specific work order and set it to repeat (Weekly for 8 occurrences, or Monthly for 12). Deleting a recurring appointment asks whether you want to remove just that one occurrence or the entire series.

8. Manual Library

Your searchable library of vehicle service manuals and repair procedures, split into two screens: the catalog browser and the reader.

Browsing — the catalog is organized in a drill-down tree: nationality group (American, Japanese, German, Korean, European, Commercial, Other) → make → year → model family → specific engine/trim. You can also filter by drivetrain or engine type, search within whatever level you're looking at, or use the separate global keyword search to jump straight to a result. A "Recently Viewed Manuals" shortcut remembers your last 6 manuals for quick return visits.

Reading — the manual reader is a two-pane view: a searchable table-of-contents tree on the left (it auto-expands and scrolls to whatever page you're on) and the actual manual content on the right. Torque specs are automatically highlighted, step-by-step procedures render as interactive checklists (note: checking items off is not saved between visits — it resets on reload), and every image opens in a full-featured lightbox with zoom, pan, rotate, print, and download. You can bookmark a manual ("Save to Garage") or save it directly to a specific vehicle's profile. Each page can be printed or downloaded as its own standalone document.

Occasionally you'll see a "Toggle {Source}" button — this appears only when two different data sources both have information for the exact same vehicle, letting you switch between them.

9. Email

Five tabs live here: Inbox, Sent Log, Templates, Compose, and Trash.

- **Inbox** — incoming email (powered by Resend), with search, date filtering, and a trash icon on each row.
- **Sent Log** — same search/filter tools, plus a delivery-status badge on each message.
- **Templates** — create, edit, or delete reusable email templates, with one-click buttons to insert placeholders like the customer's name.
- **Compose** — pick a customer (auto-fills their email if one is on file), optionally pick a template (this replaces the subject and body entirely rather than merging with what you've already typed, so pick your template first), and write the body as plain HTML in a text box — this is not a visual/WYSIWYG editor.
- **Trash** — separate Received and Sent sub-tabs; restore a message or permanently purge it. "Empty Trash" asks you to confirm, since that action can't be undone.

Opening any message gives you Reply (quotes the original) and Forward, both of which switch you into Compose with everything pre-filled. Printing a message opens a clean print-formatted view. You can also jump into Compose directly from a Customer or Job page with the relevant details already filled in.

10. Texts

Five tabs: Conversations, Inbox, Sent, Trash, and Private.

- **Conversations** — merged two-way threads, the main day-to-day view.
- **Inbox** — inbound messages only, with an unread-count badge on the tab.
- **Sent** — a flat, non-threaded chronological log of everything sent out.
- **Trash** — soft-deleted messages from both regular and Private conversations (a small lock icon marks the private ones); you can only empty the whole trash at once, there's no per-message permanent delete.
- **Private** — a separate contact rolodex, deliberately kept apart from your Customers/Jobs/Vehicles records. Good for personal or off-the-books numbers you don't want mixed into your CRM.

A connection-status pill shows whether text messaging (Twilio) is fully configured. If it isn't, a banner names the exact missing settings. In the compose box, Enter sends your message and Shift+Enter adds a line break. Opening any thread with unread messages automatically marks them read.

A green Call button appears on thread headers and Private-contact rows whenever texting is configured — clicking it bridges a real phone call: your own shop phone rings first, and once you answer, it automatically connects you to the other person.

Texts are automatically tagged by why they were sent — for example, an appointment reminder, a booking confirmation, a job-complete notice, a stale-lead follow-up, an unpaid-invoice reminder, a win-back message, or a review request — so you can tell at a glance which ones were automatic versus something you typed yourself.

11. Automations

This page is a full-screen window into your separate, self-hosted **n8n** automation tool (n8n.homeslab.uk) — it is not a settings screen inside Ragnarök itself. Building or editing an actual automation workflow (like a morning briefing text, or an automatic follow-up) happens entirely inside n8n, not on this page. The automatic text message types mentioned in the Texts section above are triggered by the app's own backend logic; there is no on/off switch for them here.

12. Payments

A read-only ledger of every Stripe payment your shop has taken. A Refresh button pulls the latest data, and four summary cards show Total Revenue, Revenue This Month, Total Transactions, and Average Transaction. You can filter by customer name, payment status, or date range, and sort by date.

Clicking a row opens a detail view with the raw Stripe session and payment-intent IDs, plus a "View Invoice" link back to the originating job.

> This page has no write actions at all — no refunds, no manually-added payments, no export button. It's a viewer only. Also note: the name search only matches customer names, not Stripe IDs or job descriptions.

13. Customer Portal (the link you send customers)

This is what a customer sees when you send them a portal link from a job ticket — it's an entirely separate, branded page with no shop navigation or login required. It shows their vehicle and contact summary, the job's current status, a before/after photo gallery, and a line-by-line breakdown of parts and services they can individually approve or decline.

> Once a customer approves or declines a line item there, there is no undo — the decision is final on that page. Make sure the ticket is accurate before sending the link.

A Pay button appears as soon as at least one item has been approved (not necessarily all of them), and takes the customer to a Stripe checkout page. Tax is calculated on parts and labor only, not on services. The page does not auto-refresh — if a customer has it open a while, they may need to manually reload to see the latest status.

14. Funnels

Funnels are single-page lead-capture landing pages, separate from your main Website(s). The list view shows each funnel's layout type, how many leads it's captured and converted, and an Embed button that gives you an iframe snippet (with specific instructions if you're embedding into Squarespace).

There are four visual layouts to choose from when building a funnel, each with its own media slots: Classic (alternating background video clips), Modern, Video (a large click-to-play hero video), and Booking (a real 3-step booking flow — pick a date, pick a time slot based on live availability, then enter contact details). Every funnel's public page includes a spam-resistant lead-capture form.

15. Websites (Site Builder)

The most feature-dense part of the app — a full drag-and-drop website builder for building a real public site for your shop (or a customer's). Note: this section of the app used to be labeled "Sites" — it's the same feature, just relabeled "Websites"/"Website Builder" for clarity.

The Sites list lets you pause or activate a site, see a live-rendered thumbnail (or set a custom one), copy or open the public link, see block/message stats, and manage site-wide Settings — name, subdomain, theme, colors, fonts, SEO meta description (with a live character counter), structured-data business type, and favicon.

Inside the builder itself:

- **Blocks tab** — add and arrange 12 block types: Hero, Text, Image Gallery, Video, Call-to-Action, Contact Form, Testimonial, Pricing, FAQ, Spacer, AI Chat Bot, Lead Funnel, and Link Button.

- **Device switcher** — preview as Desktop, Tablet, or Mobile (Mobile is a read-only stacked preview).
- **Undo/redo** — right-click the Undo or Redo button for a full history dropdown with human-readable labels; you can jump straight to any past step, not just one at a time.
- **Layers panel** — lists every block, front-most first, grouped by Locked-to-Front / Unlocked / Locked-to-Back. You can drag to reorder within a group, rename a layer, hide/show it, or delete it (with confirmation).
- **Canvas right-click menu** — Style, Zoom & Position (for media), Rotate 90° (media), Duplicate, Bring to Front/Send to Back (a one-time nudge, separate from the persistent Layers lock), Hide/Show on Mobile, and Delete.
- **Inspector panel** — a Content tab for the block's actual media/text and a Style tab for alignment, spacing, fonts, backgrounds, borders, shadows, responsive overrides, and a large bank of "Cinematic Animation" effects.
- **Site Theme tab** — 12 curated, live-rendered color/font presets to try with one click, plus manual color/font pickers; applying a preset previews instantly but still needs a separate Save Theme click to stick.
- **Export** — one dropdown button with three formats: JSON (raw data), HTML (a self-contained snapshot page), and PDF (a real browser print, chosen deliberately over more failure-prone PDF-generation libraries).
- **Templates** — 12 full starter templates with real example content, not placeholder text (Portfolio, Local Service Business, Product/App Landing Page, Personal/About Me, Restaurant/Menu, Coming Soon, Auto Repair Shop, Real Estate Listing, Fitness/Gym, Photography Studio, Nonprofit/Fundraiser, Event/Wedding). Applying one adds its blocks to the page rather than replacing what's already there.

A right-click "Zoom & Position" option is available on any image or video: scroll to zoom (100%-400%), drag to pan, with a small floating toolbar to reset or finish.

16. AI Chat Bot — Cooper & Roscoe

> Important: there are two completely different things named "AI Chat Bot" in this app. Don't confuse them.

The real assistant is the floating circular chat button in the bottom-right corner of every page (not a nav item). Click it to open a chat panel where you can ask Cooper & Roscoe questions or ask it to do things for you — look up a customer, check inventory, draft a text message, pull today's schedule, and more. It remembers the conversation only while the page stays open; refreshing the page resets it back to the greeting. Any manual diagrams it shows you open in a full lightbox with zoom, rotate, print, and download, and every reply has its own Print/Download links.

> The assistant cannot delete anything, ever, under any circumstance. If you ask it to delete a customer, job, part, or anything else, it will tell you plainly that it can't and that you need to do it yourself in the app. It also always asks you to confirm before sending a text message on your behalf.

The "AI Chat Bot" nav page is a completely separate, unrelated tool — a white-label chatbot *builder/demo* for embedding a chatbot on some other website entirely (not connected to the real Cooper & Roscoe assistant at all). It ships 20 locked persona templates you can preview, lets you configure a hypothetical bot's name, theme, and visual style (including a 3D hologram style), and exports embed code you could paste into an external site. Its "chat" preview is a local simulation only — it never talks to the real AI.

17. Video Editor

A full video editor built on a third-party tool called Twick Studio, running in its own container and embedded directly in this page. The only control this page adds of its own is "Open in New Tab," which opens the editor as a bare standalone page. When you export a video, it renders client-side and downloads automatically.

> If the Twick service happens to be down, this page currently shows a raw technical error rather than a friendly message — if you see a wall of JSON/error text here, that's what's happening; see the Troubleshooting section.

18. YouTube Trimmer

A simple wrapper around a completely separate standalone tool (its own app, its own server) for trimming YouTube videos. It has no logic of its own inside Ragnarök and doesn't hand data back to the rest of the app automatically.

19. Quick Reformat Tool

Every image or video field in the app (Website blocks, Funnels, your shop logo, thumbnails) shares the same upload widget, which always offers three ways to add media: paste a URL, upload a file directly (20MB cap for images, 100MB for video), or use the Reformat tool to shrink an oversized file first.

Reformat accepts raw files up to 2GB. Images are resized right in your browser using a simple dimension slider. Videos are re-encoded on the server with a live percentage progress bar, targeting 480p/720p/1080p output.

If a video is over 100MB and you're on the public website address, the tool will suggest an "Open Quick Uploader Locally" link (only shown if a Local/Tailscale address has been set in Settings) — this routes you around a public-internet upload size limit by using your shop's local network connection instead. See Troubleshooting section 25 for more on when and why you'd need this.

20. Manage Users (Admin Only)

Visible only to admin accounts. Three tabs: Directory (list every user, edit them, or delete them — you can't delete your own account, and deleting anyone always confirms first), Create Account (set a username, password, and role for a new user), and Re-key Passcode (as an admin, reset any user's password without needing to know their old one).

The edit screen won't let you create a duplicate username, and won't let you demote yourself out of the admin role if you're the only remaining admin account — this protects you from accidentally locking everyone out of admin access.

21. Settings

Six sections:

- **Business Diagnostics** — a live, auto-refreshing dashboard of system health. View-only, no changes made here.
- **API Server / LAN Host** — test and save the address the app uses to reach its own backend.
- **Shop Profile & Billing** — your shop's name, address, tax rate, labor rate, markup percentage, admin notification email, Google review link, Local/Tailscale access URL, owner's cell number (needed for the Call-bridge feature), and shop logo.

- **Self-Service Booking Availability** — the hours, slot length, notice period, max bookings per slot, and closed days that govern the *public* booking flow used by Funnels — this is separate from your own internal Calendar.
- **Workshop Display Theme** — 8 built-in color themes (Ragnarök Dark, Midnight Steel, Carbon Black, Arctic White, Forest Green, Blood Orange, Royal Purple, Gunmetal), applied instantly with no save step needed.
- **Custom Page Backgrounds** — set a per-page background image or video with opacity and playback-speed sliders, and a reset button to go back to default.

22. How It All Connects Together

A few relationships are worth keeping in mind as you use the app day to day:

- A Customer owns Vehicles, and a Vehicle belongs to exactly one Customer — everything else (Jobs, service history, saved manuals) hangs off one or the other of these two records.
- A Job/Work Order is where Parts (from Inventory) and Services (from your Services Catalog) actually get billed — adjusting inventory stock doesn't happen automatically from a job; you add parts to the job, which is a separate step from stock tracking.
- The Customer Portal link and the Pay Now button both originate from a single Job — there's no separate portal-management screen; it's always generated from the ticket itself.
- Funnels and Websites both capture leads, but they are separate features with separate lists — a Funnel is a single-page capture form, a Website is a full multi-block site.
- The floating AI Chat Bot (Cooper & Roscoe) can read across almost all of the above (customers, vehicles, jobs, inventory, appointments, manuals, texts) but can never delete anything and always asks before sending a text.
- Automations (the n8n page) sits outside all of this — it's a separate tool you use to build your own custom workflows against the app's data, not a feature the app manages for you automatically.

PART II

Troubleshooting Guide

This section is written for day-to-day use — plain language first, with a pointer into Part III when a problem needs a real server-side fix.

23. Login & Access Issues

Can't log in / login screen keeps reloading: confirm the app is reaching the real backend and not silently falling into the offline demo mode (look for the small floating "Log In With Real Account" button — its presence means you're in demo mode, not really logged in). If you're on the shop's local network and the public site seems unreachable, try the Local/Tailscale address instead (Settings → Shop Profile).

Whole app looks empty right after a computer restart: this is a known infrastructure issue, not lost data — see Part III, Section 37, before assuming anything was deleted.

24. Texts and Calls Not Working

No Call button, or a banner saying texting isn't configured: this means the Twilio account details aren't set up yet — this needs the actual API keys entered on the server side; you can't fix it from the Texts page itself. Ask whoever manages the server to check the required Twilio environment variables.

You're not receiving replies to texts: inbound texting depends on a webhook URL being set correctly on Twilio's own dashboard, pointed at this app's `/api/webhooks/sms` address. If that's misconfigured or was never set, replies simply never arrive — this is a one-time setup step on Twilio's side, not a bug in the app.

Call button doesn't do anything, or gives an error: make sure your own cell number is filled in under Settings → Shop Profile → owner's cell number — the Call feature calls you first, then bridges to the other person, and can't do that without your number on file. A small number of phone numbers are text-only and can't receive real calls; if that's the actual number in use, this will always fail with a Twilio error.

25. Uploads, Photos, and Video Reformatting Issues

Large video upload fails with "Failed to fetch" on the public website: this is very likely a hard size limit enforced by the network path itself (not this app), typically somewhere around 100-200MB. Use the "Open Quick Uploader Locally" link (shown automatically for videos over 100MB, if a Local/Tailscale address is set in Settings) to upload over the shop's own network instead, which has no such limit.

A specific video file fails to upload even locally: double check the file extension. Most common video formats work, but a handful of older or unusual container formats may be rejected — try re-saving/exporting the file as a standard .mp4 first.

An uploaded photo or video "disappeared" after a deploy: this was a known bug in earlier versions where uploads could be lost during a server update. It was fixed by moving all uploads to a storage location that survives updates. If

you're on a current version and still seeing this, it's worth flagging as a new, different issue rather than assuming it's the old one recurring.

26. Video Editor Errors

If the Video Editor page shows a block of raw error/JSON text instead of the editing interface, the underlying video-editing service is temporarily down. There's nothing to fix on your end — try again shortly, or use "Open in New Tab" to see if the standalone version behaves any differently.

27. Sites and Funnels Not Updating

A change you made in the Website Builder doesn't show on the live public page: changes autosave automatically after a very short delay (under a second) — if you edited something and immediately reloaded the live page, give it a moment first. If it still doesn't appear, try reopening the builder to confirm the change is actually saved before assuming it's a publishing issue.

Overlapping images/blocks look right in the editor but wrong on the live site: if you're seeing a large, unexpected gap between two blocks that were supposed to sit flush or overlap, this matches a known layout bug that has already been fixed in current versions (a spacing mismatch between the editor and the live page's grid). If you still see it, note the specific site and blocks affected.

28. Payments and Invoices Look Wrong

Remember the Payments page is read-only and pulls directly from Stripe — there's no way to manually add, edit, or refund a payment from inside this app. If a number looks wrong, check the same transaction directly in your Stripe dashboard, since that's always the source of truth. "Revenue This Month" resets on the calendar month, not a rolling 30 days, which can look like a sudden drop right at the start of a new month.

29. Data Looks Empty or Reset

If the app suddenly shows zero customers, zero jobs, or $0 everywhere right after a computer restart or a crash, this is almost always a known, already-solved infrastructure quirk where the app's storage connection goes stale — not real data loss. See Part III, Section 37 for the full explanation and fix. As a rule: don't panic and don't assume anything was deleted until that section's steps have been tried.

30. "I Pushed a Change and Nothing Happened"

This almost always means one of two things: either the automatic deploy process silently failed partway through (see Part III, Section 33), or a file that's on the "protected list" got overwritten back to an old version because it wasn't also updated in the mirror backup repo (see Part III, Section 34). Check the deploy log first before assuming the edit itself was wrong.

31. General Diagnostic Checklist

When something feels broken and you're not sure where to start, work through these in order:

- Is this the public site (workshop.homeslab.uk) or the local/Tailscale address? Try the other one — it isolates whether it's a network/proxy issue or a real app issue.
- Reload the page fully (not just re-clicking around) — several pages (Customer Portal, live Website pages) don't auto-refresh on their own.
- Check whether you're accidentally in offline/demo mode (the floating "Log In With Real Account" button is the tell).
- For anything server-side, check the container logs before assuming the problem is in the interface — most real bugs leave a clear error there.
- If data looks missing rather than just displaying oddly, go to Part III Section 37 before concluding anything was actually lost.

PART III

Technical & Service Manual

This part is written for whoever maintains the server — a future you, or anyone else picking this up cold.

32. System Architecture Overview

Frontend: React 19 + TypeScript + Vite 6 + Tailwind CSS 4. It's a single-page app — there's no react-router, just one `view` state variable in `App.tsx` that swaps between named view components.

Backend: Express 4 + better-sqlite3 (SQLite), with JWT authentication (24-hour token expiry) via jsonwebtoken and bcryptjs, mostly defined in `backend/server.js` plus a handful of split-out route files.

Key integrations:

- **Stripe** — payments/checkout (backend/stripe.js), webhook at /api/webhooks/stripe.
- **Resend** — outbound email plus an inbound email webhook with signature verification (backend/email.js).
- **Twilio** — two-way SMS and voice call bridging (backend/sms.js), inbound webhook at /api/webhooks/sms.
- **Google GenAI (Gemini)** — powers the real Cooper & Roscoe assistant (backend/chat-route.js) and the Inventory invoice parser.
- **ffmpeg** — server-side video re-encoding for the Reformat tool.
- **Twick Studio** — third-party video editor, its own Docker container, proxied through the app.

Data storage: SQLite lives at /data/db/workshop.db inside the container, bind-mounted from the repo's own data/ directory. Every file upload (job attachments, receipts, general media) is written to /data/uploads/ — the same bind mount, and for the same reason: it's the one directory that survives both a code deploy and a full container recreation.

33. The Deploy Pipeline

A GitHub webhook fires on every push to the main branch of the Workshop-Ragnarok repository. In order, it:

- Resets the working copy to exactly match what's committed on main, wiping any local drift but preserving the data/ directory and .env file.
- Restores a hardcoded list of "protected" files from a separate mirror backup repo, overwriting whatever was just reset — see Section 34.
- Rebuilds the backend Docker image.
- Stops, removes, and re-creates the backend container.
- Runs a post-rebuild script.

> The single most important rule: because the mirror restore step runs unconditionally on every deploy, you must always push the mirror (images) repo FIRST, then push Workshop-Ragnarok second. Pushing Workshop-Ragnarok first fires the deploy immediately and restores your fix right back to the old, stale mirror version before you get a chance to push it.

Routine deploys with no dependency or Dockerfile changes are fast, typically 1-2 minutes, since Docker reuses cached build layers. A full uncached rebuild (needed only when verifying a Dockerfile or dependency fix actually landed) can take 45+ minutes — that's expected, not a sign something is broken.

34. Protected Files & the Mirror Repo

There are two separate GitHub repositories involved: the actual app (Workshop-Ragnarok) and a separate "images" repo that holds trusted backup copies of a specific, hardcoded list of critical files (currently about two dozen — things like the chat widget, the Dockerfiles, package-lock files, and a set of hero video/logo assets). This mirror exists purely as an automatic restore source on every deploy, not as a general asset repository.

> Any edit to a file on the protected list must be committed to BOTH repositories to actually stick — mirror pushed first, always. This is also the entire explanation for why a protected file can appear to "keep reverting" in your git tool: the deploy webhook silently rewrites it on disk after every single deploy, independent of what's committed to Workshop-Ragnarok's own history.

35. Docker Services Reference

The backend Docker build has two stages, both on a current Node LTS base image: a frontend-builder stage that builds the Vite/React bundle, and a final runtime stage that installs only production dependencies and copies in the built frontend plus the backend source.

The compose file defines three services for this app specifically: a companion lemon-server, the workshop-backend service (the real running container is named ragnarok-backend, not workshop-backend — that's just the compose service name), and the webhook service that runs the deploy automation itself.

A .dockerignore file at the repo root excludes node_modules, .git, the data directory, build output, and markdown files from the build context — if this file is ever missing, re-add it, since every build would otherwise send hundreds of megabytes of unnecessary context.

36. Database, Backups & Uploads

The SQLite database and all uploaded files live under the repo's data/ directory, which is specifically excluded from the git-reset step of every deploy — this is what lets the database and uploaded photos/videos survive every single deploy without being wiped. A nightly backup task covers this repo's own database and upload directory specifically; it has no relationship to anything on the wider shared server described below.

37. The Shared Host Environment

This app's containers run on the same physical machine, same virtualization layer, and same Docker installation as the owner's much larger separate homelab media/services stack — a different docker-compose file entirely, unrelated to this app.

The most important recurring issue to know about: after a crash, a forced restart, or even just an ordinary computer reboot, a container can end up looking at a stale, outdated view of its own data folder, even though the real files on disk are completely untouched and intact. This shows up as a page suddenly looking empty or reset to zero. It is *not* real data loss. A plain restart of the affected container does NOT fix it, because it just resumes the same stale connection. The actual fix is a full stop-and-recreate of the container (not just a restart), which forces it to reconnect to the real, current files fresh. As of mid-2026 this has been automated with a self-healing check that runs automatically shortly after every boot, so this class of issue should now resolve itself without anyone needing to intervene by hand — if it ever doesn't, check the relevant automation's own logs first before assuming something is newly broken.

38. Historical Bug Log (Diagnosed & Fixed Issues)

A running list of real problems that have already been fully diagnosed and fixed, kept so nobody has to re-diagnose the same thing twice:

- A native-dependency install bug required a full clean reinstall of node modules; the corrected lockfile had to be pushed to both repos.
- An old Node runtime version was too outdated for several dependencies that required a newer version — fixed by upgrading the base image.
- Docker's build cache can report a step as unchanged even when the underlying file changed — always rebuild without cache when truly verifying a fix landed.
- A large, missing .dockerignore file was causing every build to send far more data than necessary — fixed by adding one.
- A recurring file-truncation issue that looked like a network problem was actually a genuinely broken file already committed at the source — always check the commit history before assuming a download/network issue.
- Large video uploads used to crash the browser tab entirely — fixed by switching the upload transport from a giant embedded text string to a proper streaming file upload.
- Very large uploads over the public internet address can fail due to a hard size cap enforced by the network/proxy layer itself, not the app — the Local/Tailscale upload path exists specifically to route around this.
- A specific rare video file format was being rejected unexpectedly — fixed by widening the list of accepted video formats to match what browsers actually report.
- A subtle deploy-tooling naming mismatch could cause a deploy to appear successful in the log while quietly failing to actually replace the running container — always check for real error lines in the deploy log, not just the final "success" line.

39. Standing Operating Rules

- Any command meant to be run manually (terminal, PowerShell, etc.) should always be given as a clearly copyable block, never as plain inline text.
- Any edit to a protected-list file must be made in both repositories, mirror pushed first, every time, no exceptions.
- When verifying whether a Docker-level fix actually worked, always rebuild without the cache — a cached build can hide the truth.
- If a file looks corrupted the exact same way across multiple different fetch attempts, suspect the file that's actually committed before suspecting the network or tooling.
- If any container on the shared host ever looks like it lost its data after a crash or forced restart, assume the stale-connection issue from Section 37 first — a full recreate, not a simple restart, is almost always the fix.

End of manual.